

Pengendalian Manual: Memberikan Keterampilan Motorik Dasar

[ Terminal: 2]

Terminal 1 (Sistem Inti):

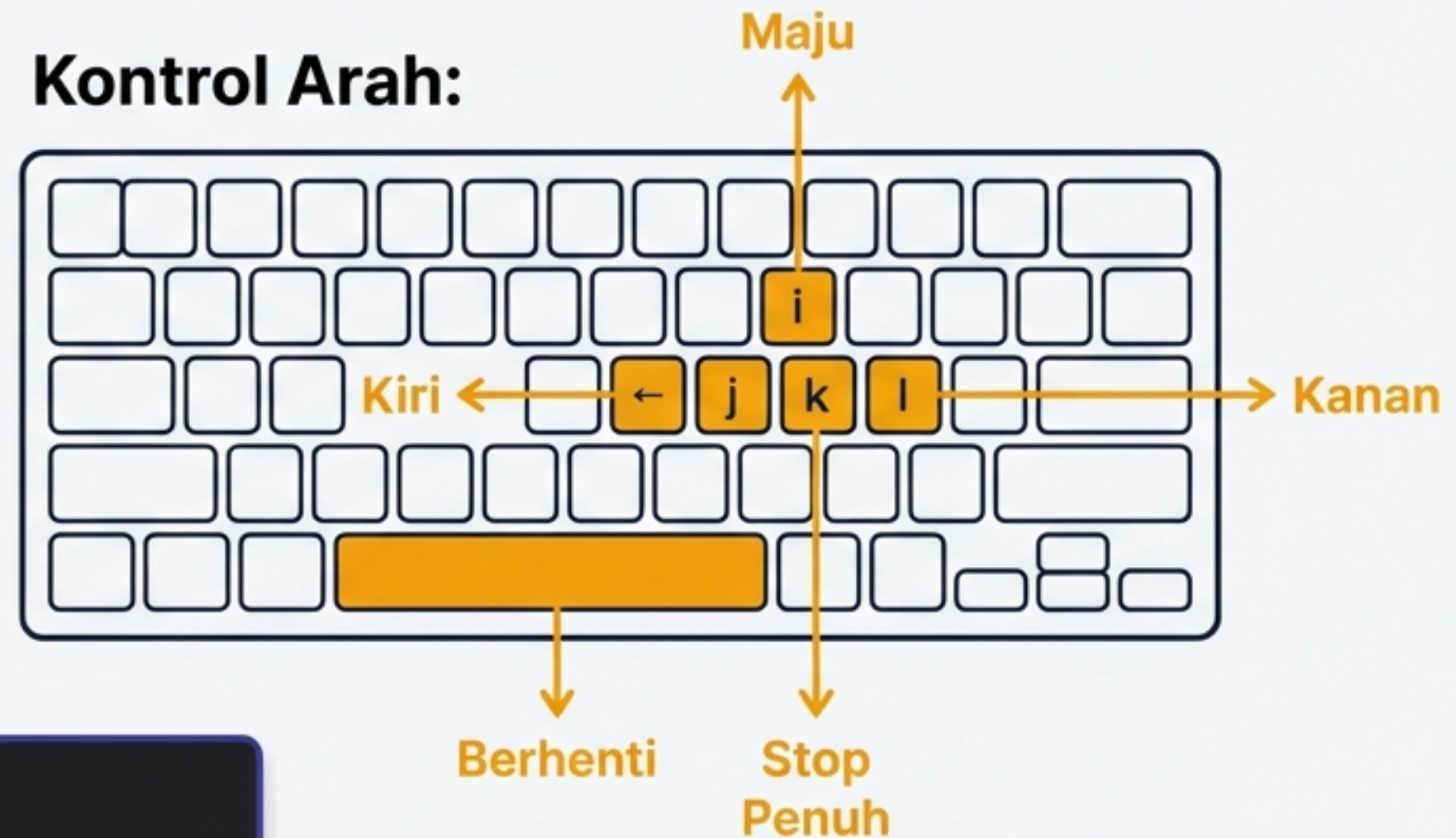
```
ros2 launch gazebo_praktikum  
percobaan6_teleop.launch.py
```

Menjalankan Gazebo, RSP, controller_manager,
diff_drive_controller, & RViz2.

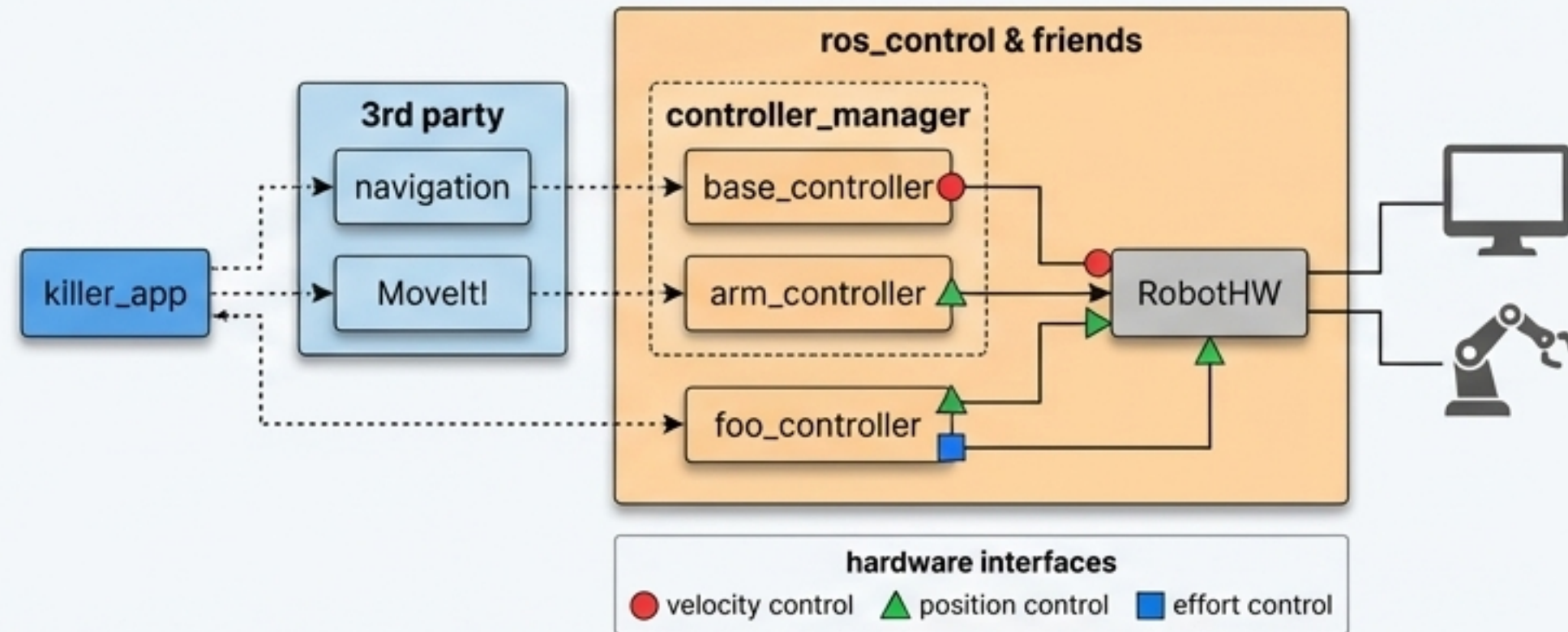
Terminal 2 (Teleoperasi):

```
ros2 run teleop_twist_keyboard  
teleop_twist_keyboard --ros-args --remap  
cmd_vel:=/diff_drive_controller/cmd_vel_unstamped
```

Kontrol Arah:



Anatomi diff_drive_controller dan Alur Transmisi Data



Input: Menerima **geometry_msgs/Twist** dari **/diff_drive_controller/cmd_vel_unstamped**

Konversi Kinematika: Mengubah nilai **linear.x** dan **angular.z** menjadi kecepatan rotasi spesifik untuk tiap roda melalui hardware interface.

Eksekusi Fisik (Wheels)

Feedback System: Mempublikasikan odometri ke **/diff_drive_controller/odom** (**nav_msgs/Odometry**) dan mengirimkan transformasi (TF) **odom** → **base_footprint** ke **/tf**.

Perintah Monitor:
`ros2 topic echo /diff_drive_controller/odom`
(Amati perubahan posisi x, y saat robot bergerak)

Membangun Ingatan Spasial dengan SLAM Toolbox

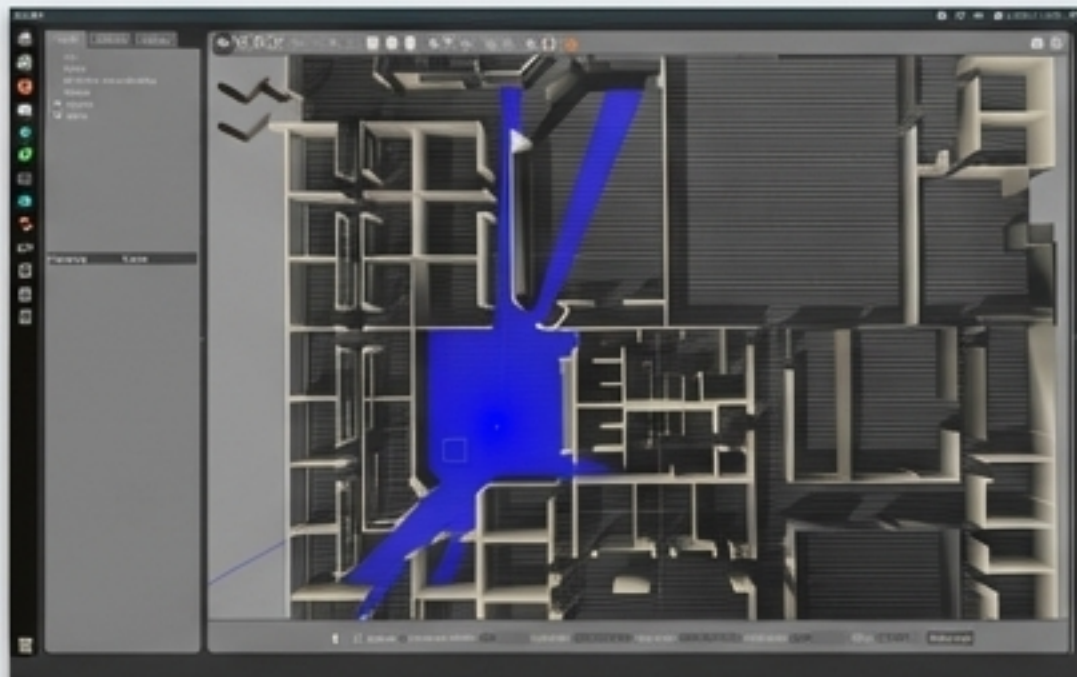
Konsep **SLAM**: Simultaneous Localization and Mapping — robot membangun peta area yang tidak diketahui sekaligus melacak posisinya di dalam peta tersebut secara real-time.

[ | **Terminal: 2**]

Langkah 1: Launch System

```
ros2 launch gazebo_praktikum  
percobaan7a_slam.launch.py
```

(Node `async_slam_toolbox_node` mulai subscribe ke `/scan` dan `/tf`).



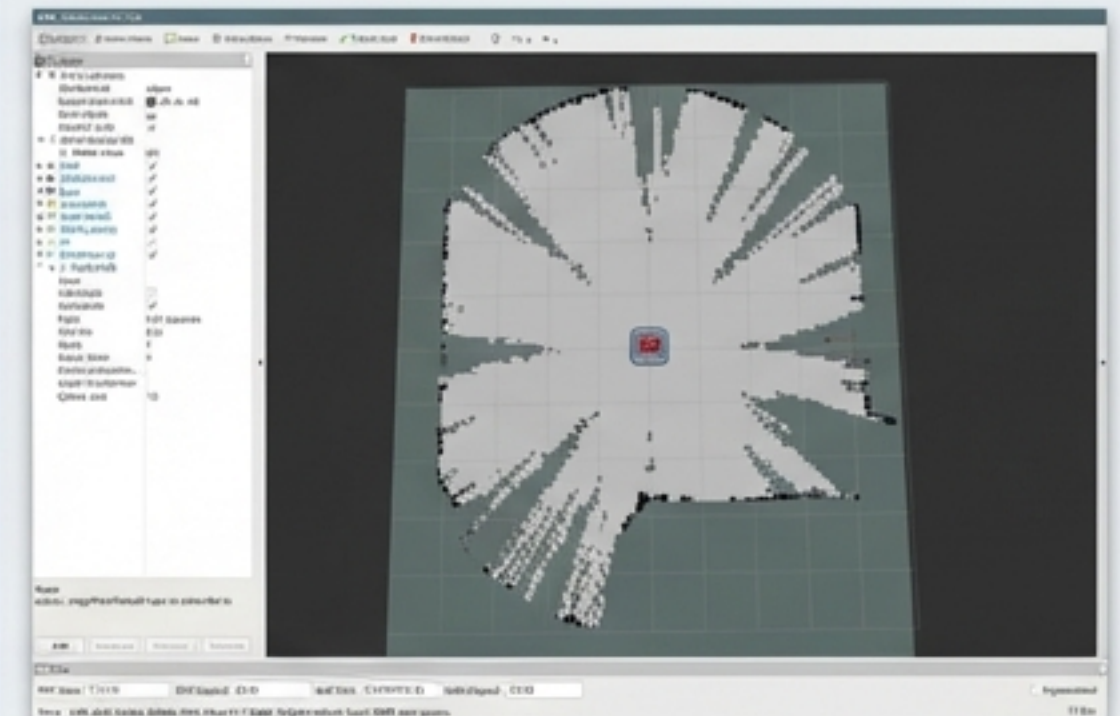
Langkah 2: Eksplorasi

Gunakan Terminal 2 (`teleop_twist_keyboard`) untuk menggerakkan robot menelusuri labirin hingga peta di panel RViz2 Map terbentuk utuh.

Langkah 3: Simpan Peta

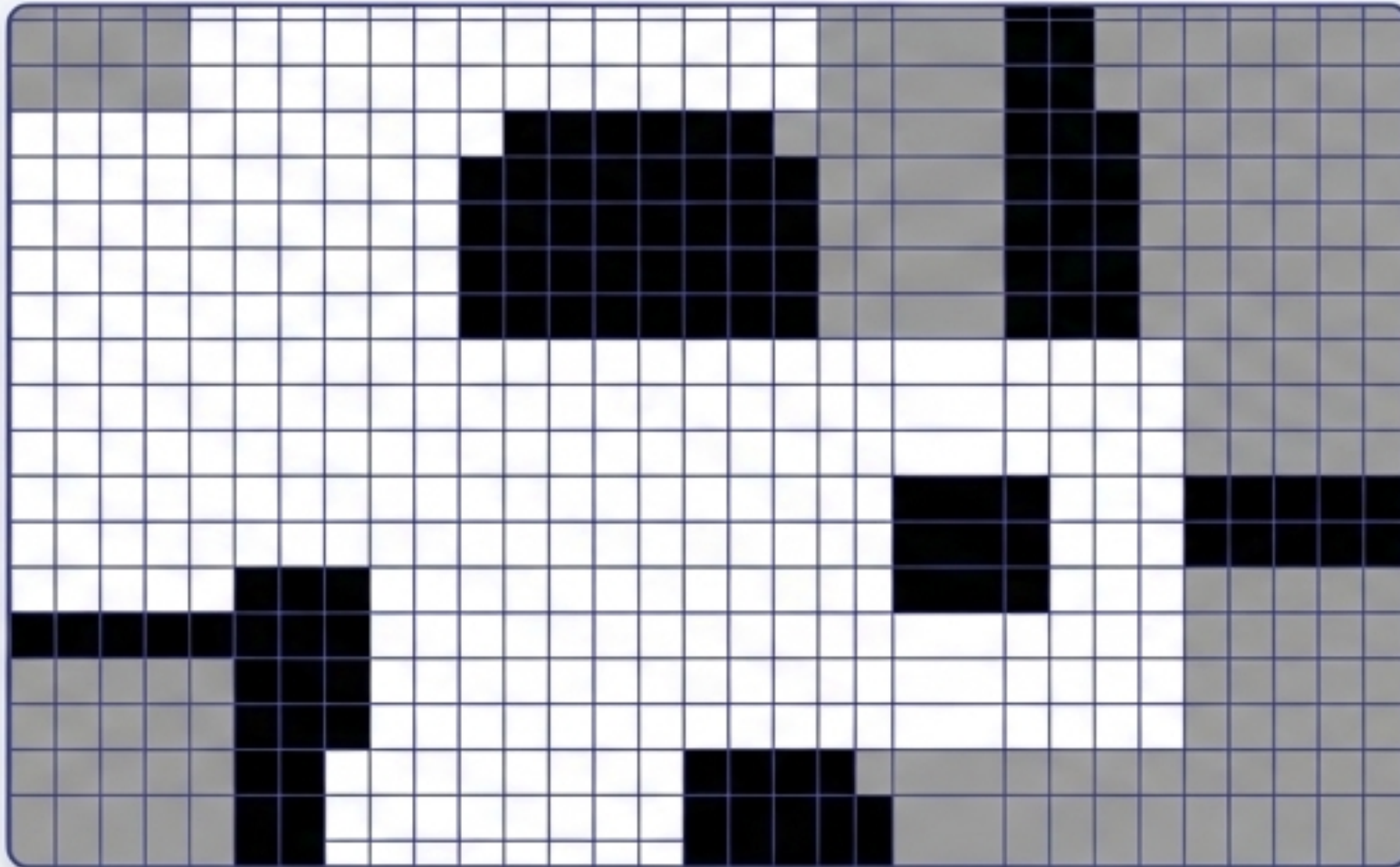
```
ros2 run nav2_map_server  
map_saver_cli -f ~/maps/my_map
```

(Membuat file `.yaml` dan `.pgm`).



Membedah Occupancy Grid & Konfigurasi Parameter SLAM

Theoretical Concept



- Format Peta (/map): Grid 2D dengan resolusi 0.05 m/sel.
 - 0 = Bebas (White)
 - 100 = Terisi / Rintangan (Black)
 - 1 = Tidak Diketahui (Grey)
- Anatomi Penyimpanan: File .yaml menyimpan metadata peta, file .pgm menyimpan gambar Occupancy Grid.

Configuration

slam_params.yaml

```
slam_toolbox:
  ros_parameters:
    # Plugin params
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_region_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS Parameters
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    mode: mapping
    map_file_name: /home/user/maps/my_map
    map_start_at_dock: true

    # Slam params
    resolution: 0.05
    max_laser_range: 20.0
    minimum_time_interval: 0.5
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: true

    # General params
    use_sim_time: true
    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02
```

Robot dalam mode pembuatan peta aktif

Tingkat detail grid

Jangkauan maksimal sensor LiDAR dalam meter

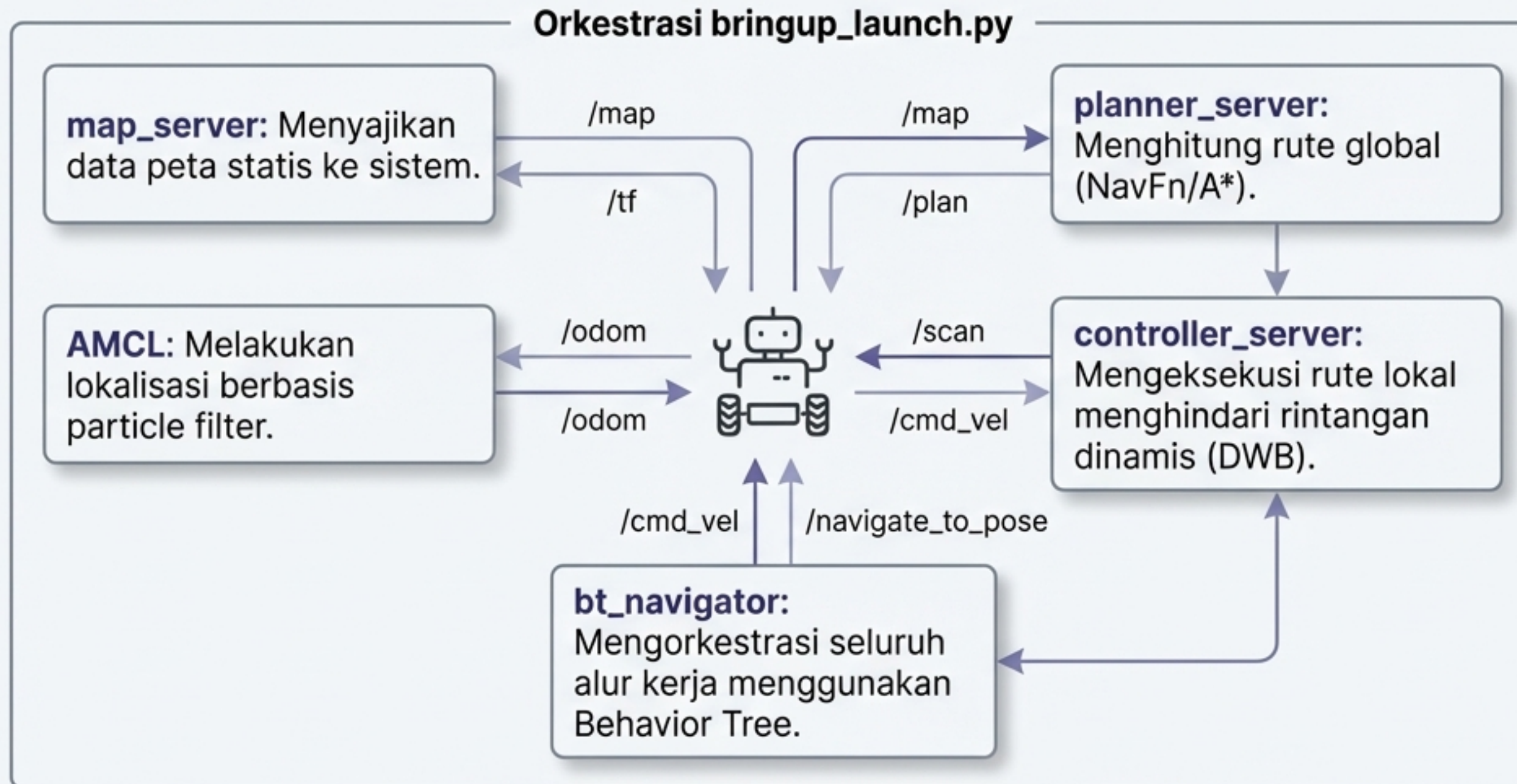
Sinkronisasi waktu dengan Gazebo

Memberikan Otak Navigasi Otonom Melalui Stack Nav2

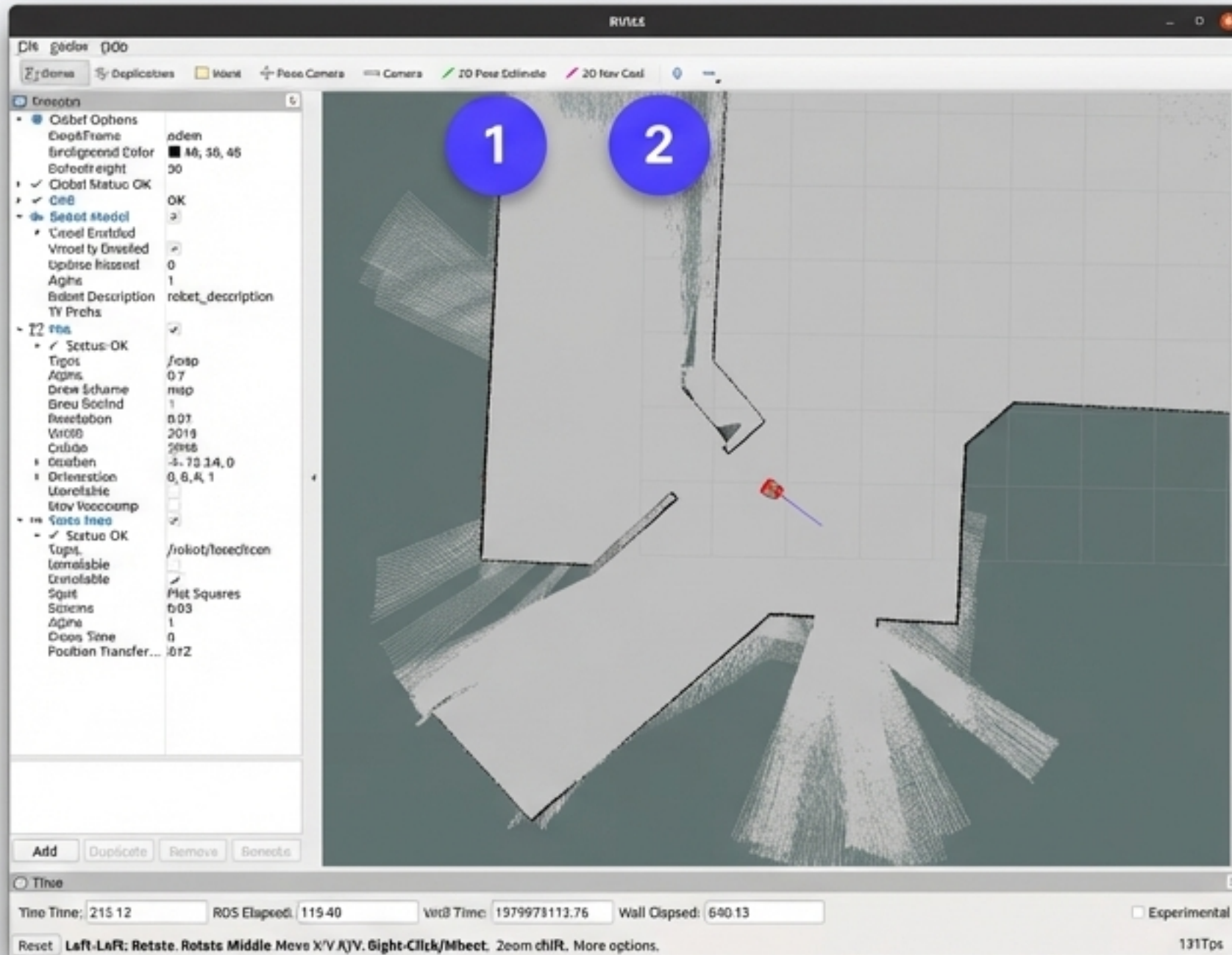
[ Terminal: 1]

```
ros2 launch gazebo_praktikum percobaan7b_navigasi.launch.py  
map:=/path/my_map.yaml
```

 Prasyarat: Membutuhkan file peta .yaml yang dihasilkan dari tahap SLAM.

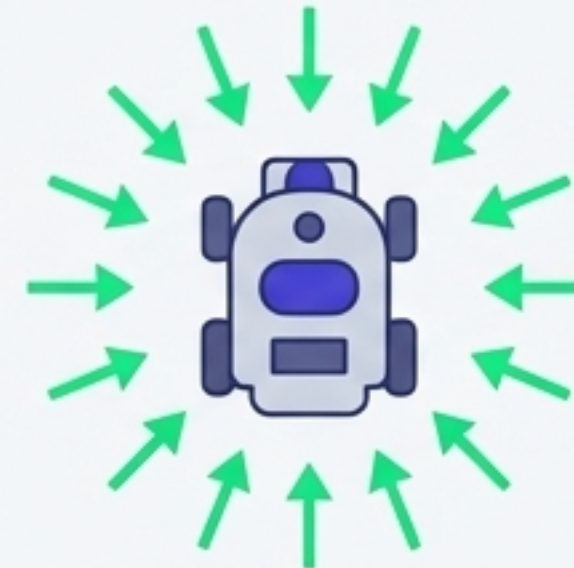


Eksekusi Navigasi: Lokalisasi AMCL di RViz2



Prosedur GUI RViz2:

1. Klik tombol **2D Pose Estimate** di toolbar atas, lalu klik posisi awal robot di peta untuk menginisialisasi partikel.
2. Klik tombol **2D Nav Goal**, lalu klik titik tujuan yang diinginkan. Robot akan merencanakan jalur dan bergerak secara otonom.

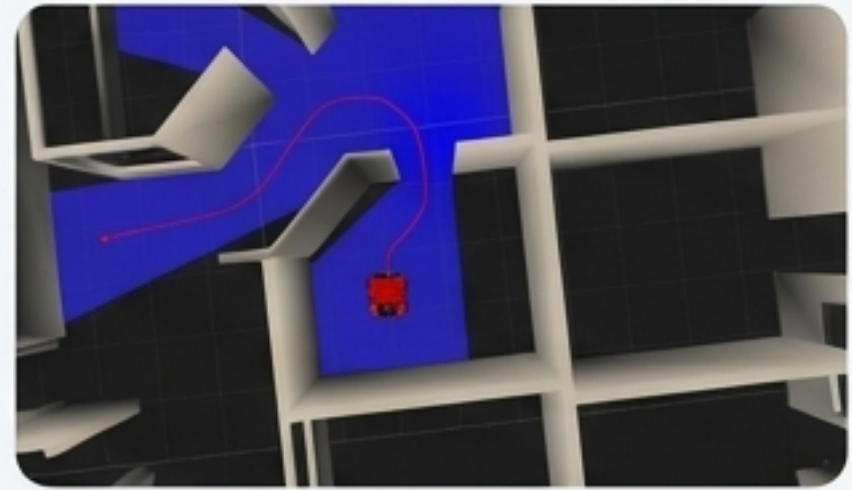


Cara Kerja AMCL:

Menggunakan particle filter dengan ratusan hipotesa posisi. Setiap scan LiDAR dibandingkan dengan peta untuk mengupdate bobot partikel hingga konvergen ke posisi aktual.

Monitor Lokalisasi: `ros2 topic echo /amcl_pose` (Melihat estimasi posisi dan orientasi secara matematis).

Tuning Perilaku Navigasi pada nav2_params.yaml



Ringkasan:

- **Lokalisasi (AMCL):** Keseimbangan antara akurasi dan beban CPU.
- **Perencanaan Jalur:** NavfnPlanner untuk rute global, DWBLocalPlanner untuk eksekusi lokal.
- **Tuning Sensitivitas:** Mengatur batas kecepatan dan area aman (costmap) di sekitar bodi robot.

```
amcl:  
  max_particles: 2000  
  min_particles: 500  
planner_server:  
  plugin: GridBased (NavfnPlanner)  
controller_server:  
  plugin: FollowPath (DWBLocalPlanner)  
  max_vel_x: 0.5  
local_costmap:  
  resolution: 0.05  
  inflation_radius: 0.55  
global_costmap:  
  resolution: 0.05
```

Turunkan nilai ini jika robot terlihat goyang (oscillation) saat berbelok tajam.

Naikkan nilai ini jika sisi bodi robot terlalu sering menyerempet dinding.

Kompleksitas Mekanis: Mengendalikan Manipulator 3-DOF

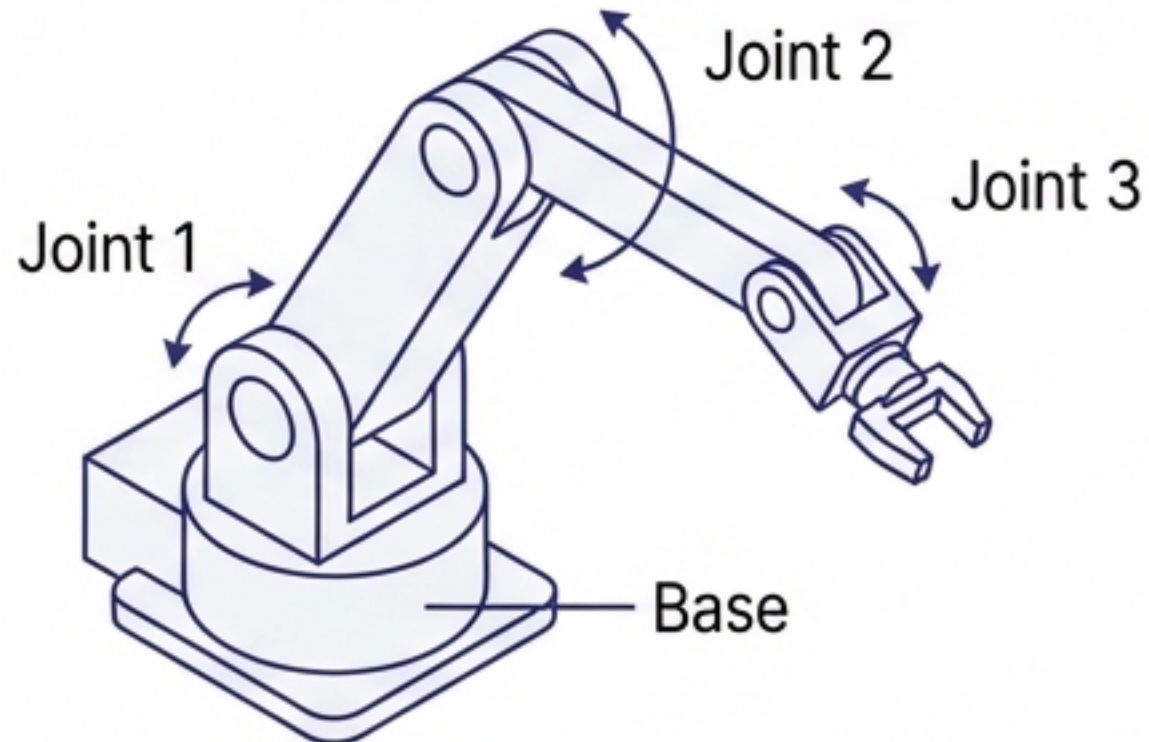
Membuka **Dimensi Baru**: Mengendalikan sistem statis dengan 3 derajat kebebasan (**Degrees of Freedom**).

 **Terminal: 1**

```
ros2 launch gazebo_praktikum percobaan8_manipulator.launch.py
```

(Script demo_manipulator.py akan otomatis berjalan setelah 8 detik).

Manajemen Status: Argumen `paused:=true` digunakan sebagai default agar controller memiliki waktu untuk bersiap sebelum simulasi fisika Gazebo berjalan. Namespace menggunakan `/manipulator/`.



Timeline Spawning (Mencegah Race Condition):

⌚ 4.0s `joint_state_broadcaster`

⌚ 5.0s `joint_1_position_controller`

⌚ 5.5s `joint_2_position_controller`

⌚ 6.0s `joint_3_position_controller`

Intervensi Manual: Perintah Sudut Manipulator

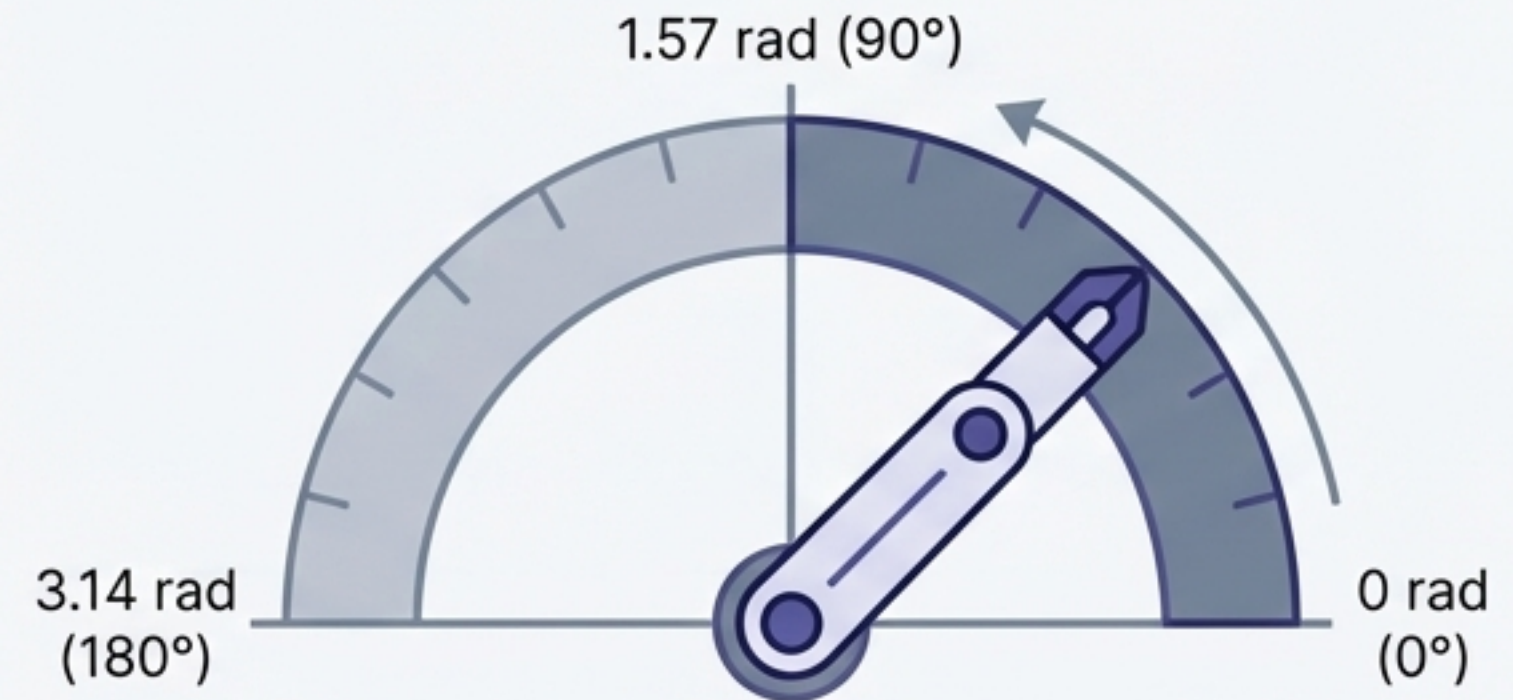
Sistem Koordinat Aktuator: Nilai target dikirimkan dalam satuan **Radian**, bukan derajat.

Publikasi Topik Manual (Terminal 2 Opsional)

```
user@robot:~$ ros2 topic pub  
/manipulator/joint_1_position_controller/command  
std_msgs/msg/Float64 'data: 1.57' --once
```

```
Target joint_2: 'data: 0.5'  
Target joint_3: 'data: -1.0'
```

Radians vs. Physical Arm Angles



Membaca Feedback Aktual:

```
ros2 topic echo /manipulator/joint_state_broadcaster/joint_states
```

Memungkinkan observasi real-time saat lengan robot bergerak mencapai posisi target di Gazebo dan RViz2.

Arsitektur Kontrol pada manipulator_controllers.yaml

Tipe Controller: Setiap joint menggunakan JointGroupPositionController.

```
type: position_controllers/JointGroupPositionController
joints: [joint_1, joint_2, joint_3]
command_interfaces: [position]
state_interfaces: [position, velocity]
```

Pengelompokan Joint: Didefinisikan secara eksplisit untuk manajemen kolektif.

Definisi Interface:
command_interfaces hanya menerima perintah posisi target.
state_interfaces membaca posisi aktual dan kecepatan pergerakan.

Sistem Spawner: Jeda waktu yang bertingkat memastikan node `controller_manager` sudah sepenuhnya siap dan memuat/memuat hardware interface sebelum spawner dipanggil untuk setiap joint.

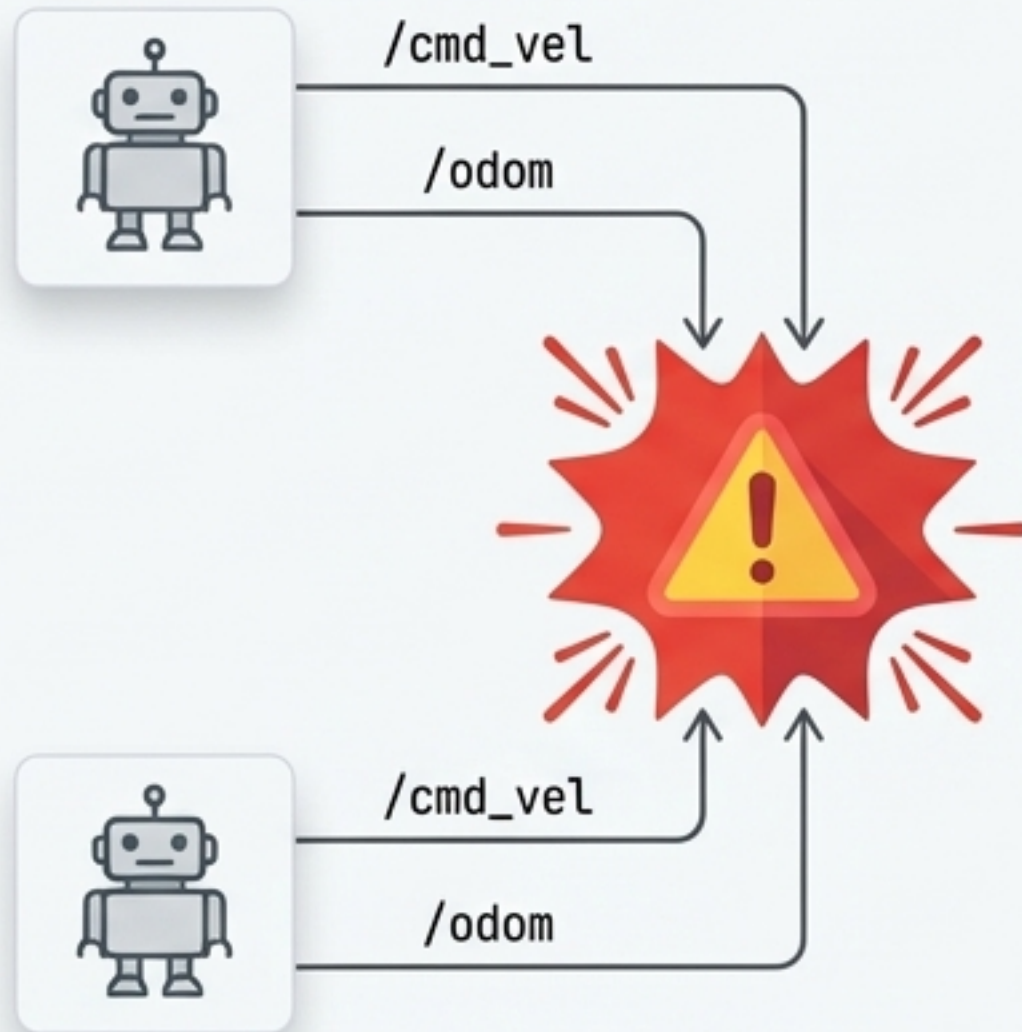
Skalabilitas Sistem: Isolasi Multi-Robot dengan Namespace

Tantangan Multi-Agen: Menjalankan dua entitas independen dalam satu simulasi tanpa konflik nama topik atau TF tree.

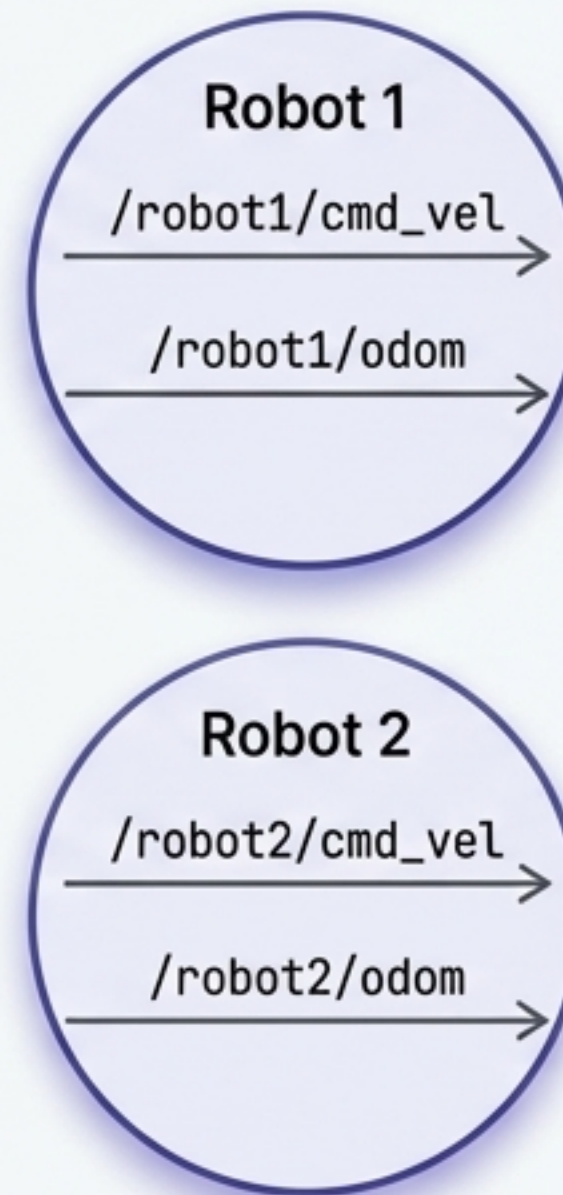
[Terminal: 3]

```
ros2 launch gazebo_praktikum  
percobaan9_multi_robot.launch.py
```

Tanpa Namespace



Dengan Namespace



Mekanisme Isolasi Node:

- Penggunaan `PushRosNamespace` dan `GroupAction` dalam script launch membungkus setiap robot dalam namespace terpisah.
- Argumen `xacro namespace:=robot1` atau `robot2` disuntikkan langsung saat spawn.
- `frame_prefix: robot1/` digunakan untuk memisahkan TF Tree agar tidak tumpang tindih.
- Geometri Awal: Robot 1 di-spawn pada $(-3, 0)$. Robot 2 di $(3, 0)$ dengan rotasi yaw 180 derajat menghadap robot pertama.

Orkestrasi Paralel: Teleoperasi Dua Robot Independen

Kendali Terpisah: Membutuhkan total 3 terminal. Terminal 1 menjalankan dunia dan robot, Terminal 2 & 3 sebagai kontroler.

Koneksi ke Robot 1 (Terminal 2)

```
ros2 run teleop_twist_keyboard  
teleop_twist_keyboard --ros-args  
--remap /cmd_vel:=/robot1/cmd_vel
```

Koneksi ke Robot 2 (Terminal 3)

```
ros2 run teleop_twist_keyboard  
teleop_twist_keyboard --ros-args  
--remap /cmd_vel:=/robot2/cmd_vel
```

Verifikasi Jaringan:

- Gunakan perintah `ros2 topic list` untuk mengonfirmasi bahwa prefix namespace (/robot1/... dan /robot2/...) terbentuk dengan benar tanpa ada tabrakan topik dasar. Keduanya dapat dikemudikan bersamaan dalam parallel streams.

Dashboard Diagnostik: Monitoring Topik & Sensor



Pemindaian LiDAR (Menghindari Spilled Output)

```
ros2 topic echo /scan --no-arr
```

Menampilkan metadata LaserScan tanpa membanjiri terminal dengan array 360 nilai jarak.



Pelacakan Posisi (Odometri)

```
ros2 topic echo /odom
```

Memantau perubahan pose robot secara kontinu.



Kesehatan Jaringan (Frekuensi & Latensi)

```
ros2 topic hz /scan  
ros2 topic delay /scan
```

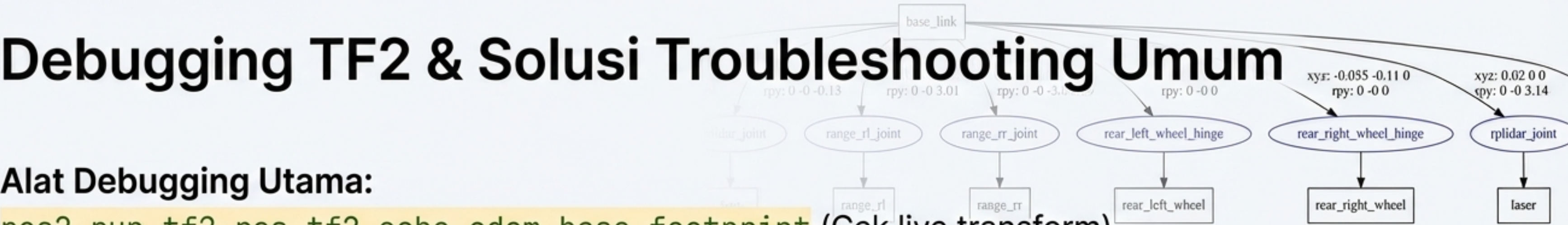
Mengecek rate data masuk dan keterlambatan transmisi sensor.



Pengecekan Visual & Relasi Node

```
ros2 run image_tools showimage --ros-args --  
remap /image:=/robot/camera/image_raw  
ros2 node info /robot_state_publisher
```

Membuka pop-up kamera dan mengaudit publisher/subscriber sistem.



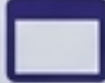
Alat Debugging Utama:

```
ros2 run tf2_ros tf2_echo odom base_footprint (Cek live transform).
ros2 run tf2_tools view_frames (Mencetak hierarki TF menjadi file PDF).
```

Panduan Resolusi Masalah

Gejala (Symptom)	Diagnosa (Diagnosis)	Solusi (Solution)
🚨 Robot tidak muncul di Gazebo	Proses spawn_entity bertabrakan.	Naikkan delay pada TimerAction di launch file.
🚨 Controller gagal di-spawn	controller_manager belum siap.	Naikkan jeda waktu (delay) berurutan pada pemanggilan spawner.
🚨 Joint lengan tidak bergerak	Hilangnya publisher state.	Pastikan node joint_state_publisher aktif berjalan.
🚨 Error 'TF not found'	Putusnya rantai transformasi.	Cek file URDF, pastikan tidak ada definisi <link> atau <joint> yang tidak tersambung ke struktur utama.

Matriks Ringkasan Eksekusi: Percobaan 1–9

Percobaan	Terminal Aktif	File Launch	Hasil Utama
P1 Empty World	1T	percobaan1_empty_world.launch.py	 UI Gazebo Dasar
P2 Objek Primitif	1T	percobaan2_shapes.launch.py	 Fisika Box/Cylinder
P3 URDF & RViz2	1T	percobaan3_urdf_rviz.launch.py	 Visualisasi TF Robot
P4 Spawn Gazebo	1T	percobaan4_spawn_robot.launch.py	 URDF masuk Simulasi
P5 Integrasi Sensor	2T	percobaan5_sensor.launch.py	 Kamera & LiDAR Aktif
P6 Kendali Teleop	2T	percobaan6_teleop.launch.py	 ros2_control Diff Drive
P7A SLAM Mapping	2T	percobaan7a_slam.launch.py	 Simpan Peta (.yaml/.pgm)
P7B Navigasi Nav2	1T	percobaan7b_navigasi.launch.py	 Otonom via RViz2
P8 Manipulator	1T	percobaan8_manipulator.launch.py	 Lengan 3-DOF Bergerak
P9 Multi-Robot	3T	percobaan9_multi_robot.launch.py	 2 Robot Namespace Terpisah